typesense

⌘ K

Ranking and Relevance

Copy Markdown

Typesense ranks search results using a simple tie-breaking sorting algorithm that can rely on one or more of:

1. Text match score, exposed as a special `_text_match` field.
2. User-defined indexed numerical fields (eg: popularity, rating, score, etc)
3. User-defined indexed string fields (eg: name) v0.23.0
4. Conditions that attribute values should match (eg: boost all records with category: shoes) v26

- Text Match Score & Type
- Tie-Breaking-based Ranking
  - Default Ranking Order
- Strict Ordering or Hard Sorting
- Ranking based on Relevance and Popularity
  - Using Fixed Number of Buckets
  - Using Fixed Bucket Size
  - Choosing Between Approaches
- Ranking based on Relevance and Recency
- Ranking exact matches
- Ranking based on conditions
- Boosting / Burying sets of records
- Promoting or Hiding Results (Merchandising)
- Tuning Typo Tolerance
- Handling No Results

Text Match Score & Type

Typesense first computes a per-field text match score based on the following heuristics:

1. **Frequency**: Number of tokens overlapping between the search query and a text field. Documents that have more overlapping tokens will be ranked above those with lesser

edit distance of num_typos characters from the query tokens. Documents that contain the tokens in the query exactly are ranked higher than those containing tokens with larger edit distances.

3. **Proximity**: Whether the query tokens appear verbatim or interspersed with other tokens in the field. Documents in which the query tokens appear right next to each other will be ranked above documents where the query tokens exist but are far apart in a text field.
4. **Ordering of `query_by` fields**: A document that matches on a field earlier in the list of `query_by` fields is considered more relevant than a document matched on a field later in the list. This implicit weighting can be overrided with `query_by_weights`.
5. **Field weights specified in `query_by_weights` field**: A document that matches a field with a higher score is considered more relevant than a document that matches on a field with a lower score. These weights override the implicit weights from ordering of `query_by` fields.

Per-field text match scores are then used to arrive at a per-document aggregated text match score that's used to order the documents when you set the `sort_by` search parameter to `_text_match:desc`.

Given that different search use cases will require different strategies, Typesense supports several text matching modes for the computation of the aggregated per-document score. This can be configured via the `text_match_type` search parameter, and it can contain the following values:

- `max_score` (default): The largest text match score across all fields is picked as the representative score of the document. Field weights are used only as a tie-breaker when 2 documents share the same text match score. In this mode, we give priority to a field matching the words in the query as much as possible. Only when several fields match the query equally, we use the field weights to decide which document should be prioritized.
- `max_weight`: The text match score of the highest weighted field is used as the representative score of the record. In this mode, we give priority to matches on the fields that we deem to be most important. In doing so, a document which matches the query partially on a field with higher weightage is deemed to be more relevant than a document that matches the query completely on a field with lesser weightage.
- `sum_score`. We sum the field-level weighted text match scores to arrive at a document-level score. In this mode, we consider the contribution of all fields to a document's overall match with the query. The downside of this mode is that a document with partial matches on several low-weighted fields can be prioritized over a document with a complete match on a single higher weighted field.

Tie-Breaking-based Ranking

There might be cases when many documents contain the exact tokens in a search query. In such a case, their `_text_match` will also be the same. That's when the user-defined indexed numerical and string fields can be used to break the tie. You can specify up to two such user-defined fields to use for ranking.

For example, let's say that we're searching for books with a query like `short story`. If there are multiple books containing these exact words, then all those documents will have the same text match score.

To break the tie, we could specify up to two additional `sort_by` fields. For instance, we could say:

```json
{
  "sort_by": "_text_match:desc,average_rating:desc,publication_year:desc"
}
```

This would sort the results in the following manner:

1. All matching records are sorted by their text match score.
2. If any two document share the same text match score, sort them by average rating.
3. If there is still a tie, sort them by their year of publication.

Default Ranking Order

When you don't provide a `sort_by` parameter to your search request, the documents will be ranked in the following order:

1. First on the _text_match score
2. Then default sorting field values specified in the collection's schema
3. And finally, if not specified the document insertion order.

Strict Ordering or Hard Sorting

If you wish to sort the documents strictly by an indexed numerical or string field like `price`, `name`, etc, you can just move the text match score criteria to the end as follows:

```
}
```

Ranking based on Relevance and Popularity

If you have a popularity score for your documents that you have either:

1. calculated on your end in your application using any formula of your choice or
2. calculated using a counter analytics rule in Typesense

You can have Typesense mix your custom scores with the text relevance score it calculates, so results that are more popular (as defined by your custom score) are boosted more in ranking. There are two approaches you can use:

Using Fixed Number of Buckets

Here's how to divide results into a specific number of relevance groups:

```json
{
  "sort_by": "_text_match(buckets: 10):desc,weighted_score:desc"
}
```

Where `weighted_score` is a field in your document with your custom score.

This will do the following:

1. Fetch all results matching the query
2. Sort them by text relevance (text match score desc)
3. Divide the results into equal-sized 10 buckets (with the first bucket containing the most relevant results)
4. Force all results within each bucket to tie in `_text_match` score. So all results in the 1st bucket will be forced to have the same text match score, all results in the 2nd bucket will be forced to have the same text match score, etc.
5. This will cause a tie inside each bucket, and then the `weighted_score` will be used to break the tie and re-rank results within each bucket.

The higher the number of buckets, the more granular the re-ranking based on your weighted score will be. For example, if you have 100 results, and `buckets: 50`, then each bucket will have 2

Using Fixed Bucket Size

Alternatively, you can group results into fixed-size relevance groups:

```json
{
  "sort_by": "_text_match(bucket_size: 3):desc,weighted_score:desc"
}
```

This approach will:

1. Fetch all results matching the query
2. Sort them by text relevance
3. Group results into fixed-size buckets (e.g., 3 results per bucket)
4. Apply the `weighted_score` sorting within each fixed-size bucket

For example, if you have 100 results and `bucket_size: 3`, Typesense will create approximately 33 buckets with 3 results each. Each group of 3 results with similar text relevance will be sorted by their `weighted_score`.

Choosing Between Approaches

- Use `buckets` when you want a specific number of relevance groups
- Use `bucket_size` when you want to ensure a consistent number of results are compared by popularity at a time
- Both approaches help you balance between text relevance and popularity in your search results

Ranking based on Relevance and Recency

A common need is to rank results have been published recently higher than older results.

To implement this in Typesense, you want to store the **Unix timestamp** when a document was published as an **int64** field (eg: `published_at_timestamp`).

Now to sort based on both text relevance and recency, you could use the following `sort_by` parameter:

```
}
```

This will do the following:

1. Fetch all results matching the query
2. Sort them by text relevance (text match score desc)
3. Divide the results into equal-sized 10 buckets (with the first bucket containing the most relevant results)
4. Force all results within each bucket to tie in `_text_match` score. So all results in the 1st bucket will be forced to have the same text match score, all results in the 2nd bucket will be forced to have the same text match score, etc.
5. This will cause a tie inside each bucket, and then the `published_at_timestamp` will be used to break the tie and re-rank results within each bucket.

The higher the number of buckets, the more granular the re-ranking based on your weighted score will be. For eg, if you have 100 results, and `buckets: 50`, then each bucket will have 2 results, those two results within each bucket will be re-ranked based on your `published_at_timestamp`.

Ranking exact matches

By default, if the search query matches a particular field value verbatim, Typesense deems that document to have the highest relevancy and prioritizes it.

However, there might be cases where this might not be desirable behavior. You can turn this off by setting `prioritize_exact_match=false` when searching.

Ranking based on conditions

You can rank documents based on any expressions that evaluate to either `true` or `false`, using the special `_eval(<expression>)` operation as a `sort_by` parameter.

The syntax for the expression inside `_eval()` is the same as the `filter_by` search parameter, so we also call this feature "Optional Filtering".

For eg:

```
}
```

This will result in documents where `in_stock` is set to `true` to be ranked first, followed by documents where `in_stock` is set to `false`.

Boosting / Burying sets of records

Instead of sorting on just `true / false` values like above, we can also provide custom scores to the records matching a bunch of filter clauses.

For example, if we have a `shoes` collection and if we wish to rank all `Nike` shoes ahead of `Adidas` shoes, we can do:

```json
{
  "sort_by": "_eval([ (brand:Nike):3, (brand:Adidas):2 ]):desc"
}
```

There can be as many expressions as needed in the `_eval` and each of those expressions can be as complex as standard `filter_by` expressions.

Promoting or Hiding Results (Merchandising)

You can choose to pin (or hide) particular records by ID in particular ranking positions:

1. Based on a search query by setting up Overrides (aka Curation) or
2. Dynamically using the `pinned_hits` and `hidden_hits` search parameter

For eg: if someone searches for `phone`, you could setup an override to pin a particular product that has a good deal on it in position 1.

Another common use-case for pinning results is merchandising in an ecommerce store, where a merchandiser (or your own custom ML model) might want to curate the exact products that should appear next to each other for a given product category. Depending on the category page that the user is viewing, you can use the `pinned_hits` parameter to define which records should show up in which position for that page.

category page is being rendered.

Tuning Typo Tolerance

Typesense handles typographical errors automatically for you out-of-the-box. But there might be use cases where you might need to turn off typo tolerance or tweak its sensitivity (eg: part numbers, phone numbers)

To turn off typo tolerance completely, set `num_typos=0` and `typo_tokens_threshold=0` when searching.

You can also tweak typo tolerance sensitivity by setting those values to higher numbers as needed. To control typo tolerance based on word length, you can use the `min_len_1typo` and `min_len_2typo` search parameters.

You can also adjust typo tolerance settings for individual fields by specifying multiple comma-separated values for `num_typos` . For eg: if you have `query_by=name,phone_number,zip_code` and you don't want typo tolerance on `phone_number` or `zip_code` you can set `num_typos=2,0,0` .

Handling No Results

In some use-cases, if all of the user's search terms don't match any of the documents, you might not want to show the user a "No results found" message.

In such cases, you can have Typesense automatically drop words / tokens from the user's search query, one at a time and repeat the search to show results that are close to the user's original query.

This behavior is controlled by the `drop_tokens_threshold` search parameter, which has a default value of `1` . This means that if a search query only returns 1 or 0 results, Typesense will start dropping search keywords and repeat the search until at least 1 result is found.

To turn this behavior off, set `drop_tokens_threshold=0`

Edit page

Last Updated: 1/31/2025, 2:01:04 AM

typesense

⌘ K